

Class 2

COMPLETE

Using Jupyter and Pandas

```
In [1]: # Import pandas module
import pandas as pd
```

1. Importing Data

To use Pandas, we need data - we can make our own data, or we can import it.

1.1 Using a **numpy** array of your data

Here is an example of creating a Pandas DataFrame from a numpy array

```
In [2]: # import numpy then make a numpy array
import numpy as np
res = np.array([[350,300],[1,1],[9,6],[12,16]])
res
```

```
Out[2]: array([[350, 300],
               [  1,   1],
               [  9,   6],
               [ 12,  16]])
```

```
In [3]: # convert numpy array to DataFrame using pd.DataFrame
# provide list of column and (optionally) row names through 'columns' and 'index' arguments
res_df = pd.DataFrame(res,columns=['AquaSpas','HydroLuxes'],index=['profit','pumps used',
res_df
```

```
Out[3]:
```

	AquaSpas	HydroLuxes
--	----------	------------

	AquaSpas	HydroLuxes
profit	350	300
pumps used	1	1
labor used	9	6
tubing used	12	16

1.1 Making data with a dict

Here's an example of making data from a dictionary and turning it into a pandas DataFrame

```
In [4]: # Make PFE data using a dictionary.
# The key is a string which will be the column headers in our data frame.
# The value is a list of strings which will give us our rows in the data frame
# Leave Code in***
pfe_data = {'first_name': ['Casey', 'John', 'Suzie', 'Joe'], 'last_name': ['Williams',
pfe_data
```

```
Out[4]: {'first_name': ['Casey', 'John', 'Suzie', 'Joe'],
'last_name': ['Williams', 'Miller', 'Smith', 'Lee'],
'fall': [250, 275, 190, 205],
'spring': [270, 265, 200, 210]}
```

```
In [5]: # extract the keys for use as column names
colNames = pfe_data.keys()
colNames
```

```
Out[5]: dict_keys(['first_name', 'last_name', 'fall', 'spring'])
```

```
In [6]: # Make a dataframe object and view it
# Recall that dicts are not stored in any particular order. By using the columns para
# specify the order in which to show the dataframe columns if need be
pfe_df = pd.DataFrame(pfe_data, columns=colNames)
pfe_df
```

```
Out[6]:
```

	first_name	last_name	fall	spring
0	Casey	Williams	250	270
1	John	Miller	275	265
2	Suzie	Smith	190	200
3	Joe	Lee	205	210

1.2 Importing data from a csv file

You'll need to specify the path of where the csv file is, or make sure it's in the same folder that this file is in.

```
In [7]: # We will import a csv file that contains all NYC 311 Service Requests from 2010 to pr
# (This data has been subset to only include Agency = NYPD)
df = pd.read_csv('311_from_2011.csv')

# Note that because this is a much larger dataset than what we created, the "preview"
# is only the first 30 rows and the last 30 rows
df
```

Out[7]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	
0	20874484	7/21/2011 2:34	7/21/2011 3:18	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	
1	20874524	7/21/2011 9:53	7/22/2011 7:56	NYPD	New York City Police Department	Homeless Encampment	NaN	
2	20874653	7/21/2011 9:04	7/21/2011 13:14	NYPD	New York City Police Department	Illegal Parking	Posted Parking Sign Violation	
3	20875098	7/21/2011 9:46	7/21/2011 12:47	NYPD	New York City Police Department	Illegal Parking	Double Parked Blocking Vehicle	
4	20875294	7/21/2011 15:03	7/21/2011 17:59	NYPD	New York City Police Department	Illegal Parking	Double Parked Blocking Traffic	
...	...	...	...	...	...	...	...	
152615	22335139	12/15/2011 10:31	12/15/2011 14:10	NYPD	New York City Police Department	Blocked Driveway	Partial Access	
152616	22335145	12/15/2011 15:55	12/15/2011 18:44	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	
152617	22335147	12/16/2011 0:48	12/16/2011 1:19	NYPD	New York City Police Department	Noise - Vehicle	Engine Idling	
152618	22335152	12/16/2011 0:16	12/16/2011 6:07	NYPD	New York City Police Department	Noise - Commercial	Loud Music/Party	S
152619	22335153	12/16/2011 1:45	12/16/2011 3:02	NYPD	New York City Police Department	Noise - Commercial	Loud Music/Party	Clu

152620 rows × 53 columns



```
In [8]: # To read about this method (or other methods/functions) and its parameters
#       type ? and then the method name
```

?pd.read_csv

Signature:

```
pd.read_csv(  
    filepath_or_buffer: 'FilePath | ReadCsvBuffer[bytes] | ReadCsvBuffer[str]',  
    *,  
    sep: 'str | None | lib.NoDefault' = <no_default>,  
    delimiter: 'str | None | lib.NoDefault' = None,  
    header: "int | Sequence[int] | None | Literal['infer']" = 'infer',  
    names: 'Sequence[Hashable] | None | lib.NoDefault' = <no_default>,  
    index_col: 'IndexLabel | Literal[False] | None' = None,  
    usecols=None,  
    dtype: 'DtypeArg | None' = None,  
    engine: 'CSVEngine | None' = None,  
    converters=None,  
    true_values=None,  
    false_values=None,  
    skipinitialspace: 'bool' = False,  
    skiprows=None,  
    skipfooter: 'int' = 0,  
    nrows: 'int | None' = None,  
    na_values=None,  
    keep_default_na: 'bool' = True,  
    na_filter: 'bool' = True,  
    verbose: 'bool' = False,  
    skip_blank_lines: 'bool' = True,  
    parse_dates: 'bool | Sequence[Hashable] | None' = None,  
    infer_datetime_format: 'bool | lib.NoDefault' = <no_default>,  
    keep_date_col: 'bool' = False,  
    date_parser=<no_default>,  
    date_format: 'str | None' = None,  
    dayfirst: 'bool' = False,  
    cache_dates: 'bool' = True,  
    iterator: 'bool' = False,  
    chunksize: 'int | None' = None,  
    compression: 'CompressionOptions' = 'infer',  
    thousands: 'str | None' = None,  
    decimal: 'str' = '.',  
    lineterminator: 'str | None' = None,  
    quotechar: 'str' = '"',  
    quoting: 'int' = 0,  
    doublequote: 'bool' = True,  
    escapechar: 'str | None' = None,  
    comment: 'str | None' = None,  
    encoding: 'str | None' = None,  
    encoding_errors: 'str | None' = 'strict',  
    dialect: 'str | csv.Dialect | None' = None,  
    on_bad_lines: 'str' = 'error',  
    delim_whitespace: 'bool' = False,  
    low_memory=True,  
    memory_map: 'bool' = False,  
    float_precision: "Literal['high', 'legacy'] | None" = None,  
    storage_options: 'StorageOptions' = None,  
    dtype_backend: 'DtypeBackend | lib.NoDefault' = <no_default>,  
) -> 'DataFrame | TextFileReader'
```

Docstring:

Read a comma-separated values (csv) file into DataFrame.

Also supports optionally iterating or breaking of the file into chunks.

Additional help can be found in the online docs for
`IO Tools <[https://pandas.pydata.org/pandas-docs/stable/user\\_guide/io.html](https://pandas.pydata.org/pandas-docs/stable/user_guide/io.html)>`_.

Parameters

`filepath_or_buffer` : str, path object or file-like object

Any valid string path is acceptable. The string could be a URL. Valid URL schemes include http, ftp, s3, gs, and file. For file URLs, a host is expected. A local file could be: `file://localhost/path/to/table.csv`.

If you want to pass in a path object, pandas accepts any ```os.PathLike```.

By file-like object, we refer to objects with a ```read()``` method, such as a file handle (e.g. via builtin ```open``` function) or ```StringIO```.

`sep` : str, default `','`

Delimiter to use. If `sep` is `None`, the C engine cannot automatically detect the separator, but the Python parsing engine can, meaning the latter will be used and automatically detect the separator by Python's builtin sniffer tool, ```csv.Sniffer```. In addition, separators longer than 1 character and different from ```'\s+'``` will be interpreted as regular expressions and will also force the use of the Python parsing engine. Note that regex delimiters are prone to ignoring quoted data. Regex example: ```'\r\t'```.

`delimiter` : str, default ```None```

Alias for `sep`.

`header` : int, list of int, None, default `'infer'`

Row number(s) to use as the column names, and the start of the data. Default behavior is to infer the column names: if no names are passed the behavior is identical to ```header=0``` and column names are inferred from the first line of the file, if column names are passed explicitly then the behavior is identical to ```header=None```. Explicitly pass ```header=0``` to be able to replace existing names. The header can be a list of integers that specify row locations for a multi-index on the columns e.g. `[0,1,3]`. Intervening rows that are not specified will be skipped (e.g. 2 in this example is skipped). Note that this parameter ignores commented lines and empty lines if ```skip_blank_lines=True```, so ```header=0``` denotes the first line of data rather than the first line of the file.

`names` : array-like, optional

List of column names to use. If the file contains a header row, then you should explicitly pass ```header=0``` to override the column names. Duplicates in this list are not allowed.

`index_col` : int, str, sequence of int / str, or False, optional, default ```None```

Column(s) to use as the row labels of the ```DataFrame```, either given as string name or column index. If a sequence of int / str is given, a `MultiIndex` is used.

Note: ```index_col=False``` can be used to force pandas to *not* use the first column as the index, e.g. when you have a malformed file with delimiters at

the end of each line.

`usecols` : list-like or callable, optional

Return a subset of the columns. If list-like, all elements must either be positional (i.e. integer indices into the document columns) or strings that correspond to column names provided either by the user in ``names`` or inferred from the document header row(s). If ``names`` are given, the document header row(s) are not taken into account. For example, a valid list-like ``usecols`` parameter would be ``[0, 1, 2]`` or ``['foo', 'bar', 'baz']``. Element order is ignored, so ``usecols=[0, 1]`` is the same as ``[1, 0]``. To instantiate a DataFrame from ``data`` with element order preserved use ``pd.read_csv(data, usecols=['foo', 'bar'])[['foo', 'bar']]`` for columns in ``['foo', 'bar']`` order or ``pd.read_csv(data, usecols=['foo', 'bar'])[['bar', 'foo']]`` for ``['bar', 'foo']`` order.

If callable, the callable function will be evaluated against the column names, returning names where the callable function evaluates to True. An example of a valid callable argument would be ``lambda x: x.upper() in ['AAA', 'BBB', 'DDD']``. Using this parameter results in much faster parsing time and lower memory usage.

`dtype` : Type name or dict of column -> type, optional

Data type for data or columns. E.g. `{ 'a': np.float64, 'b': np.int32, 'c': 'Int64' }`

Use ``str`` or ``object`` together with suitable ``na_values`` settings to preserve and not interpret dtype.

If converters are specified, they will be applied INSTEAD of dtype conversion.

.. versionadded:: 1.5.0

Support for `defaultdict` was added. Specify a `defaultdict` as input where the default determines the dtype of the columns which are not explicitly listed.

`engine` : `{ 'c', 'python', 'pyarrow' }`, optional

Parser engine to use. The C and pyarrow engines are faster, while the python engine is currently more feature-complete. Multithreading is currently only supported by the pyarrow engine.

.. versionadded:: 1.4.0

The "pyarrow" engine was added as an *experimental* engine, and some features are unsupported, or may not work correctly, with this engine.

`converters` : dict, optional

Dict of functions for converting values in certain columns. Keys can either be integers or column labels.

`true_values` : list, optional

Values to consider as True in addition to case-insensitive variants of "True".

`false_values` : list, optional

Values to consider as False in addition to case-insensitive variants of "False".

`skipinitialspace` : bool, default False

Skip spaces after delimiter.

`skiprows` : list-like, int or callable, optional

Line numbers to skip (0-indexed) or number of lines to skip (int) at the start of the file.

If callable, the callable function will be evaluated against the row indices, returning True if the row should be skipped and False otherwise. An example of a valid callable argument would be ``lambda x: x in [0, 2]``.

`skipfooter` : int, default 0
Number of lines at bottom of file to skip (Unsupported with engine='c').

`nrows` : int, optional
Number of rows of file to read. Useful for reading pieces of large files.

`na_values` : scalar, str, list-like, or dict, optional
Additional strings to recognize as NA/NaN. If dict passed, specific per-column NA values. By default the following values are interpreted as NaN: `'', '#N/A', '#N/A N/A', '#NA', '-1.#IND', '-1.#QNAN', '-NaN', '-nan', '1.#IND', '1.#QNAN', '<NA>', 'N/A', 'NA', 'NULL', 'NaN', 'None', 'n/a', 'nan', 'null'`.

`keep_default_na` : bool, default True
Whether or not to include the default NaN values when parsing the data. Depending on whether ``na_values`` is passed in, the behavior is as follows:

- * If ``keep_default_na`` is True, and ``na_values`` are specified, ``na_values`` is appended to the default NaN values used for parsing.
- * If ``keep_default_na`` is True, and ``na_values`` are not specified, only the default NaN values are used for parsing.
- * If ``keep_default_na`` is False, and ``na_values`` are specified, only the NaN values specified ``na_values`` are used for parsing.
- * If ``keep_default_na`` is False, and ``na_values`` are not specified, no strings will be parsed as NaN.

Note that if ``na_filter`` is passed in as False, the ``keep_default_na`` and ``na_values`` parameters will be ignored.

`na_filter` : bool, default True
Detect missing value markers (empty strings and the value of `na_values`). In data without any NAs, passing `na_filter=False` can improve the performance of reading a large file.

`verbose` : bool, default False
Indicate number of NA values placed in non-numeric columns.

`skip_blank_lines` : bool, default True
If True, skip over blank lines rather than interpreting as NaN values.

`parse_dates` : bool or list of int or names or list of lists or dict, default False
The behavior is as follows:

- * boolean. If True -> try parsing the index.
- * list of int or names. e.g. If `[1, 2, 3]` -> try parsing columns 1, 2, 3 each as a separate date column.
- * list of lists. e.g. If `[[1, 3]]` -> combine columns 1 and 3 and parse as a single date column.
- * dict, e.g. `{'foo' : [1, 3]}` -> parse columns 1, 3 as date and call result 'foo'

If a column or index cannot be represented as an array of datetimes, say because of an unparsable value or a mixture of timezones, the column or index will be returned unaltered as an object data type. For non-standard datetime parsing, use ``pd.to_datetime`` after ``pd.read_csv``.

Note: A fast-path exists for iso8601-formatted dates.

`infer_datetime_format` : bool, default False
 If True and ``parse_dates`` is enabled, pandas will attempt to infer the format of the datetime strings in the columns, and if it can be inferred, switch to a faster method of parsing them. In some cases this can increase the parsing speed by 5-10x.

.. deprecated:: 2.0.0
 A strict version of this argument is now the default, passing it has no effect.

`keep_date_col` : bool, default False
 If True and ``parse_dates`` specifies combining multiple columns then keep the original columns.

`date_parser` : function, optional
 Function to use for converting a sequence of string columns to an array of datetime instances. The default uses ``dateutil.parser.parser`` to do the conversion. Pandas will try to call ``date_parser`` in three different ways, advancing to the next if an exception occurs: 1) Pass one or more arrays (as defined by ``parse_dates``) as arguments; 2) concatenate (row-wise) the string values from the columns defined by ``parse_dates`` into a single array and pass that; and 3) call ``date_parser`` once for each row using one or more strings (corresponding to the columns defined by ``parse_dates``) as arguments.

.. deprecated:: 2.0.0
 Use ``date_format`` instead, or read in as ``object`` and then apply `:func:`to_datetime`` as-needed.

`date_format` : str or dict of column -> format, default ``None``
 If used in conjunction with ``parse_dates``, will parse dates according to this format. For anything more complex, please read in as ``object`` and then apply `:func:`to_datetime`` as-needed.

.. versionadded:: 2.0.0

`dayfirst` : bool, default False
 DD/MM format dates, international and European format.

`cache_dates` : bool, default True
 If True, use a cache of unique, converted dates to apply the datetime conversion. May produce significant speed-up when parsing duplicate date strings, especially ones with timezone offsets.

`iterator` : bool, default False
 Return `TextFileReader` object for iteration or getting chunks with ``get_chunk()``.

.. versionchanged:: 1.2
``TextFileReader`` is a context manager.

`chunksize` : int, optional
 Return `TextFileReader` object for iteration.
 See the ``IO Tools docs``
<https://pandas.pydata.org/pandas-docs/stable/io.html#io-chunking> for more information on ``iterator`` and ``chunksize``.

.. versionchanged:: 1.2

`TextFileReader` is a context manager.

`compression` : str or dict, default 'infer'

For on-the-fly decompression of on-disk data. If 'infer' and 'filepath_or_buffer' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression).

If using 'zip' or 'tar', the ZIP file must contain only one data file to be read in.

Set to `None` for no decompression.

Can also be a dict with key `'method'` set to one of `{'zip', 'gzip', 'bz2', 'zstd', 'tar'}` and other key-value pairs are forwarded to `zipfile.ZipFile`, `gzip.GzipFile`, `bz2.BZ2File`, `zstandard.ZstdDecompressor` or `tarfile.TarFile`, respectively.

As an example, the following could be passed for Zstandard decompression using a custom compression dictionary:

```
compression={'method': 'zstd', 'dict_data': my_compression_dict}
```

.. versionadded:: 1.5.0
Added support for `.tar` files.

.. versionchanged:: 1.4.0 Zstandard support.

`thousands` : str, optional
Thousands separator.

`decimal` : str, default '.'
Character to recognize as decimal point (e.g. use ',' for European data).

`lineterminator` : str (length 1), optional
Character to break file into lines. Only valid with C parser.

`quotechar` : str (length 1), optional
The character used to denote the start and end of a quoted item. Quoted items can include the delimiter and it will be ignored.

`quoting` : int or csv.QUOTE_* instance, default 0
Control field quoting behavior per `csv.QUOTE_*` constants. Use one of `QUOTE_MINIMAL` (0), `QUOTE_ALL` (1), `QUOTE_NONNUMERIC` (2) or `QUOTE_NONE` (3).

`doublequote` : bool, default `True`
When `quotechar` is specified and `quoting` is not `QUOTE_NONE`, indicate whether or not to interpret two consecutive `quotechar` elements INSIDE a field as a single `quotechar` element.

`escapechar` : str (length 1), optional
One-character string used to escape other characters.

`comment` : str, optional
Indicates remainder of line should not be parsed. If found at the beginning of a line, the line will be ignored altogether. This parameter must be a single character. Like empty lines (as long as `skip_blank_lines=True`), fully commented lines are ignored by the parameter `header` but not by `skiprows`. For example, if `comment='#'`, parsing `#empty\na,b,c\n1,2,3` with `header=0` will result in 'a,b,c' being treated as the header.

`encoding` : str, optional, default "utf-8"
Encoding to use for UTF when reading/writing (ex. 'utf-8'). `List of Python`

standard encodings

<<https://docs.python.org/3/library/codecs.html#standard-encodings>>`_ .

.. versionchanged:: 1.2

When ``encoding`` is ``None``, ``errors="replace"`` is passed to ``open()``. Otherwise, ``errors="strict"`` is passed to ``open()``. This behavior was previously only the case for ``engine="python"``.

.. versionchanged:: 1.3.0

``encoding\_errors`` is a new argument. ``encoding`` has no longer an influence on how encoding errors are handled.

encoding_errors : str, optional, default "strict"

How encoding errors are treated. `List of possible values

<<https://docs.python.org/3/library/codecs.html#error-handlers>>`_ .

.. versionadded:: 1.3.0

dialect : str or csv.Dialect, optional

If provided, this parameter will override values (default or not) for the following parameters: `delimiter`, `doublequote`, `escapechar`, `skipinitialspace`, `quotechar`, and `quoting`. If it is necessary to override values, a ParserWarning will be issued. See csv.Dialect documentation for more details.

on_bad_lines : {'error', 'warn', 'skip'} or callable, default 'error'

Specifies what to do upon encountering a bad line (a line with too many fields).

Allowed values are :

- 'error', raise an Exception when a bad line is encountered.
- 'warn', raise a warning when a bad line is encountered and skip that line.
- 'skip', skip bad lines without raising or warning when they are encountered.

.. versionadded:: 1.3.0

.. versionadded:: 1.4.0

- callable, function with signature
``(bad\_line: list[str]) -> list[str] | None`` that will process a single bad line. ``bad\_line`` is a list of strings split by the ``sep``. If the function returns ``None``, the bad line will be ignored. If the function returns a new list of strings with more elements than expected, a ``ParserWarning`` will be emitted while dropping extra elements. Only supported when ``engine="python"``

delim_whitespace : bool, default False

Specifies whether or not whitespace (e.g. ``' '`` or ``'\t'``) will be used as the sep. Equivalent to setting ``sep='\s+'``. If this option is set to True, nothing should be passed in for the ``delimiter`` parameter.

low_memory : bool, default True

Internally process the file in chunks, resulting in lower memory use while parsing, but possibly mixed type inference. To ensure no mixed

types either set False, or specify the type with the `dtype` parameter. Note that the entire file is read into a single DataFrame regardless, use the `chunks` or `iterator` parameter to return the data in chunks. (Only valid with C parser).

memory_map : bool, default False

If a filepath is provided for `filepath\_or\_buffer`, map the file object directly onto memory and access the data directly from there. Using this option can improve performance because there is no longer any I/O overhead.

float_precision : str, optional

Specifies which converter the C engine should use for floating-point values. The options are ``None`` or 'high' for the ordinary converter, 'legacy' for the original lower precision pandas converter, and 'round_trip' for the round-trip converter.

.. versionchanged:: 1.2

storage_options : dict, optional

Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to ``urllib.request.Request`` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to ``fsspec.open``. Please see ``fsspec`` and ``urllib`` for more details, and for more examples on storage options refer [here](https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files) <https://pandas.pydata.org/docs/user_guide/io.html?highlight=storage_options#reading-writing-remote-files>`_.

.. versionadded:: 1.2

dtype_backend : {"numpy_nullable", "pyarrow"}, defaults to NumPy backed DataFrames

Which dtype_backend to use, e.g. whether a DataFrame should have NumPy arrays, nullable dtypes are used for all dtypes that have a nullable implementation when "numpy_nullable" is set, pyarrow is used for all dtypes if "pyarrow" is set.

The dtype_backends are still experimental.

.. versionadded:: 2.0

Returns

DataFrame or TextFileReader

A comma-separated values (csv) file is returned as two-dimensional data structure with labeled axes.

See Also

DataFrame.to_csv : Write DataFrame to a comma-separated values (csv) file.

read_csv : Read a comma-separated values (csv) file into DataFrame.

read_fwf : Read a table of fixed-width formatted lines into DataFrame.

Examples

```
>>> pd.read_csv('data.csv') # doctest: +SKIP
```

File: c:\users\ifrommer\.conda\envs\infosys\lib\site-packages\pandas\io\parsers\re

aders.py
Type: function

2. Inspecting the data

You can see this is a large dataset. Instead we may choose to look at the first few rows or last few rows using the `head()` and `tail()` methods

2.1 Looking at the first / last few rows

```
In [9]: # To Look at the first few rows  
df.head()
```

Out[9]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Location Type
0	20874484	7/21/2011 2:34	7/21/2011 3:18	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	Street/Sidewalk
1	20874524	7/21/2011 9:53	7/22/2011 7:56	NYPD	New York City Police Department	Homeless Encampment	NaN	Street/Sidewalk
2	20874653	7/21/2011 9:04	7/21/2011 13:14	NYPD	New York City Police Department	Illegal Parking	Posted Parking Sign Violation	Street/Sidewalk
3	20875098	7/21/2011 9:46	7/21/2011 12:47	NYPD	New York City Police Department	Illegal Parking	Double Parked Blocking Vehicle	Street/Sidewalk
4	20875294	7/21/2011 15:03	7/21/2011 17:59	NYPD	New York City Police Department	Illegal Parking	Double Parked Blocking Traffic	Street/Sidewalk

5 rows × 53 columns



```
In [10]: # To Look at the Last few rows  
df.tail()
```

Out[10]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	
152615	22335139	12/15/2011 10:31	12/15/2011 14:10	NYPD	New York City Police Department	Blocked Driveway	Partial Access	
152616	22335145	12/15/2011 15:55	12/15/2011 18:44	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	
152617	22335147	12/16/2011 0:48	12/16/2011 1:19	NYPD	New York City Police Department	Noise - Vehicle	Engine Idling	
152618	22335152	12/16/2011 0:16	12/16/2011 6:07	NYPD	New York City Police Department	Noise - Commercial	Loud Music/Party	Sto
152619	22335153	12/16/2011 1:45	12/16/2011 3:02	NYPD	New York City Police Department	Noise - Commercial	Loud Music/Party	Club

5 rows × 53 columns



2.2 Summary of the data

We can also see a summary of the data by using info() method

```
In [13]: # If we want a summary of df
df.info(show_counts=True)
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 152620 entries, 0 to 152619

Data columns (total 53 columns):

#	Column	Non-Null Count	Dtype
0	Unique Key	152620 non-null	int64
1	Created Date	152620 non-null	object
2	Closed Date	152584 non-null	object
3	Agency	152620 non-null	object
4	Agency Name	152620 non-null	object
5	Complaint Type	152620 non-null	object
6	Descriptor	149800 non-null	object
7	Location Type	152617 non-null	object
8	Incident Zip	152602 non-null	float64
9	Incident Address	120132 non-null	object
10	Street Name	120132 non-null	object
11	Cross Street 1	120337 non-null	object
12	Cross Street 2	120316 non-null	object
13	Intersection Street 1	32035 non-null	object
14	Intersection Street 2	32035 non-null	object
15	Address Type	152394 non-null	object
16	City	152616 non-null	object
17	Landmark	231 non-null	object
18	Facility Type	152616 non-null	object
19	Status	152620 non-null	object
20	Due Date	152618 non-null	object
21	Resolution Description	146828 non-null	object
22	Resolution Action Updated Date	152616 non-null	object
23	Community Board	152620 non-null	object
24	Borough	152620 non-null	object
25	X Coordinate (State Plane)	152479 non-null	float64
26	Y Coordinate (State Plane)	152479 non-null	float64
27	Park Facility Name	152620 non-null	object
28	Park Borough	152620 non-null	object
29	School Name	152620 non-null	object
30	School Number	152620 non-null	object
31	School Region	152620 non-null	object
32	School Code	152620 non-null	object
33	School Phone Number	152620 non-null	object
34	School Address	152620 non-null	object
35	School City	152620 non-null	object
36	School State	152620 non-null	object
37	School Zip	152620 non-null	object
38	School Not Found	152617 non-null	object
39	School or Citywide Complaint	0 non-null	float64
40	Vehicle Type	0 non-null	float64
41	Taxi Company Borough	0 non-null	float64
42	Taxi Pick Up Location	0 non-null	float64
43	Bridge Highway Name	226 non-null	object
44	Bridge Highway Direction	226 non-null	object
45	Road Ramp	210 non-null	object
46	Bridge Highway Segment	210 non-null	object
47	Garage Lot Name	0 non-null	float64
48	Ferry Direction	0 non-null	float64

```

49 Ferry Terminal Name      0 non-null      float64
50 Latitude                152479 non-null float64
51 Longitude               152479 non-null float64
52 Location                152479 non-null object
dtypes: float64(12), int64(1), object(40)
memory usage: 61.7+ MB

```

3. Selecting rows / columns

To select row(s) or column(s) we can use the index operator []

```

In [14]: # Selecting rows 7-8
df[7:9]

```

```

Out[14]:

```

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Location Type
7	20875507	7/21/2011 14:06	7/21/2011 14:22	NYPD	New York City Police Department	Illegal Parking	Posted Parking Sign Violation	Street/Sidewalk
8	20875542	7/21/2011 15:06	7/22/2011 11:47	NYPD	New York City Police Department	Derelict Vehicle	With License Plate	Street/Sidewalk

2 rows × 53 columns



```

In [15]: # Select a column by name
df['Complaint Type']

```

```

Out[15]:
0      Illegal Parking
1    Homeless Encampment
2      Illegal Parking
3      Illegal Parking
4      Illegal Parking
...
152615  Blocked Driveway
152616  Illegal Parking
152617  Noise - Vehicle
152618  Noise - Commercial
152619  Noise - Commercial
Name: Complaint Type, Length: 152620, dtype: object

```

```

In [16]: # Selecting a column by name, and only showing the first 5 rows
df['Complaint Type'][:5]
# df[:5]['Complaint Type'] results are the same

```

```
Out[16]: 0      Illegal Parking
          1      Homeless Encampment
          2      Illegal Parking
          3      Illegal Parking
          4      Illegal Parking
          Name: Complaint Type, dtype: object
```

```
In [17]: # Selecting several columns by name gives a new pandas dataframe
          df[['Complaint Type', 'Descriptor']][:3]
```

```
Out[17]:
```

	Complaint Type	Descriptor
0	Illegal Parking	Blocked Sidewalk
1	Homeless Encampment	NaN
2	Illegal Parking	Posted Parking Sign Violation

4. Column Operations

4.1 Sorting columns

We may be interested in sorting the dataframe based off a particular column. We can do this in ascending or descending order.

```
In [18]: df.sort_values(by='Created Date', ascending=True)
```

Out[18]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Loca
5818	19503332	1/1/2011 0:01	1/1/2011 0:21	NYPD	New York City Police Department	Noise - Commercial	Loud Music/Party	Store/C
5809	19502937	1/1/2011 0:09	1/1/2011 8:19	NYPD	New York City Police Department	Blocked Driveway	No Access	Stree
5836	19504859	1/1/2011 0:21	1/1/2011 2:27	NYPD	New York City Police Department	Blocked Driveway	Partial Access	Stree
5810	19503050	1/1/2011 0:23	1/1/2011 4:53	NYPD	New York City Police Department	Noise - Commercial	Loud Music/Party	Club/Bar/
5814	19503087	1/1/2011 0:26	1/1/2011 3:28	NYPD	New York City Police Department	Blocked Driveway	Partial Access	Stree
...	...	...	...	...	...	...	...	...
110814	21398163	9/9/2011 9:52	9/9/2011 15:19	NYPD	New York City Police Department	Derelict Vehicle	With License Plate	Stree
110829	21398301	9/9/2011 9:56	9/9/2011 11:47	NYPD	New York City Police Department	Illegal Parking	Commercial Overnight Parking	Stree
110759	21397458	9/9/2011 9:57	9/9/2011 13:18	NYPD	New York City Police Department	Blocked Driveway	No Access	Stree
110743	21397219	9/9/2011 9:57	9/9/2011 10:27	NYPD	New York City Police Department	Derelict Vehicle	With License Plate	Stree
110823	21398223	9/9/2011 9:59	9/9/2011 12:06	NYPD	New York City Police Department	Blocked Driveway	No Access	Stree

152620 rows × 53 columns



4.2 Creating a new column

We also may want to add a column to our dataframe. Possibly from a combination of our other columns or maybe all "hard coded" with some number or string.

```
In [19]: df['Complaint'] = df['Complaint Type'] + df['Descriptor']
df[:1]
```

Out[19]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Location Type
0	20874484	7/21/2011 2:34	7/21/2011 3:18	NYPD	New York City Police Department	Illegal Parking	Blocked Sidewalk	Street/Sidewalk

1 rows × 54 columns

4.3 Unique values in a column

To get an array of all unique values in a particular column, use the `.unique()` method

```
In [20]: df['Incident Zip'].unique()
```

```
Out[20]: array([11372., 10014., 10040., 11216., 10027., 10034., 11355., 10016.,
11426., 11212., 11236., 11220., 11434., 10465., 11423., 10467.,
10036., 11378., 11229., 10452., 10312., 11230., 10013., 10466.,
10472., 11207., 11421., 11435., 10017., 11368., 11103., 10025.,
10307., 11416., 11375., 11223., 10021., 11218., 10023., 10460.,
11208., 11418., 11206., 11377., 10305., 10462., 11224., 11357.,
11214., 10031., 11234., 10309., 11356., 10458., 11211., 11413.,
11210., 11004., 11361., 10029., 11427., 10455., 10010., 11419.,
11105., 10002., 10461., 10009., 11374., 10456., 11231., 10314.,
11417., 11225., 11102., 11433., 10012., 10018., 11379., 11213.,
10007., 11217., 10280., 10306., 10019., 11693., 11385., 11432.,
10468., 11235., 11222., 11221., 10003., 10032., 11219., 11691.,
10024., 10004., 10308., 10038., 11354., 11203., 11101., 11226.,
11365., 11106., 11420., 11436., 10469., 11428., 11232., 11238.,
10464., 10457., 11204., 11373., 11215., 11367., 11370., 11237.,
11369., 10473., 11422., 10033., 11694., 11415., 11429., 10451.,
10039., 10301., 10304., 10065., 11209., 11364., 11233., 10006.,
10128., 10474., 10454., 11412., 11363., 11104., 10035., 10459.,
11411., 10302., 10303., 11692., 11366., 11414., 10011., 10026.,
10001., 11205., 11358., 10310., 10075., 10022., 11228., 10463.,
10030., 10453., 10028., 10000., 11109., 10037., 11201., 10470.,
11239., 11360., 10475., 10471., 11362., 10005., 11040., 10020.,
11001., 10069., 10282., 11243., 11430., nan, 10281., 10048.,
11697., 10400., 10044., 11371.]])
```

4.4 Column Statistics

You can isolate a column and find the max, min, sum of a column. For example, we may be interested in finding the zipcode shows up the most in this dataset...i.e. we are looking for the

mode.

```
In [21]: from statistics import mode

mode(df['Incident Zip'])  # or df['Incident Zip'].mode() and no need for the import
```

Out[21]: 11211.0

5. Counting values

5.1 Counting values in a given column

If we are curious how many times each value appears in the dataset, we can use the `value_counts()` method

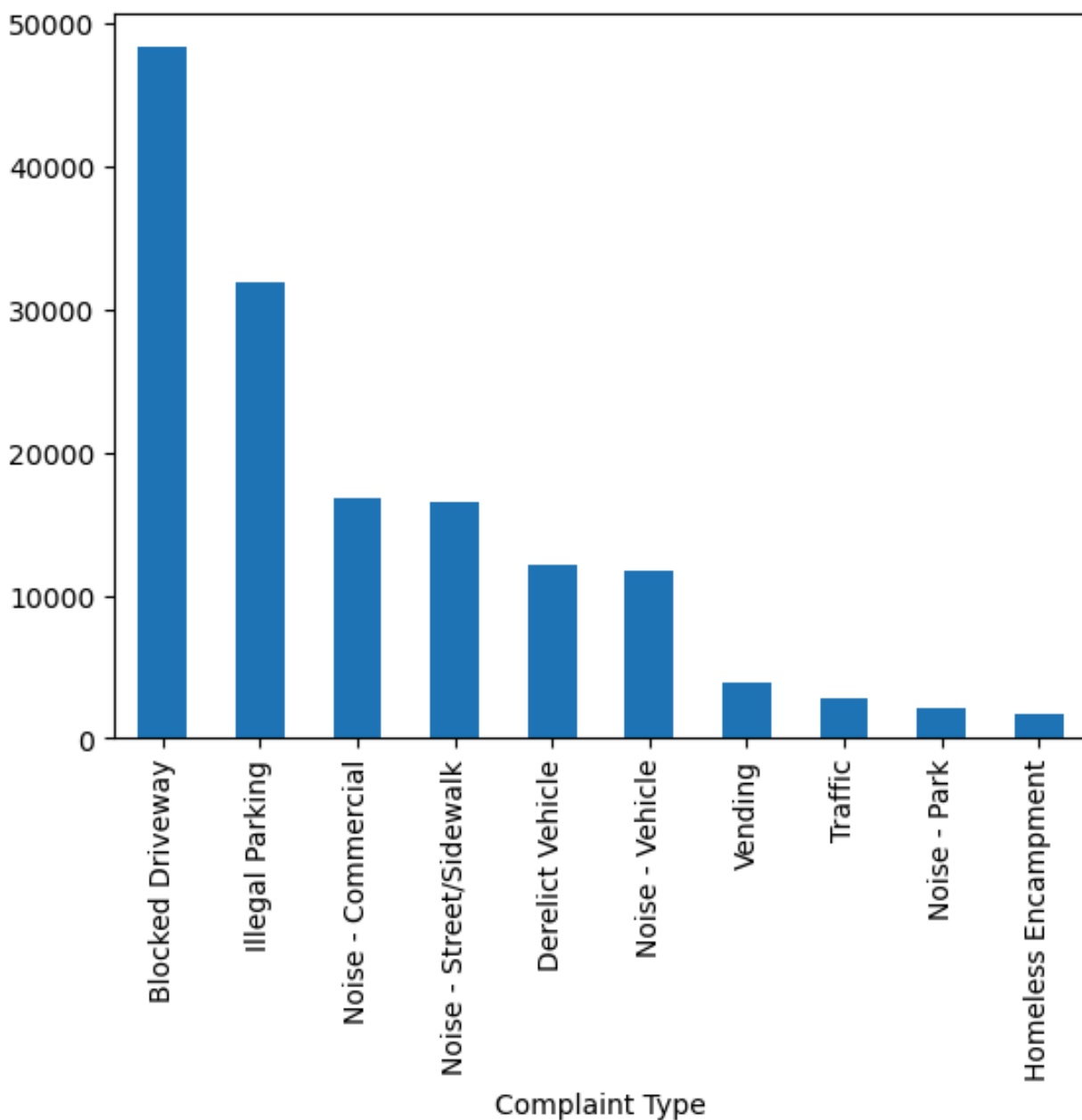
```
In [22]: # This gets the top 10 complaint types
df['Complaint Type'].value_counts()[:10]
```

```
Out[22]: Complaint Type
Blocked Driveway          48260
Illegal Parking           31867
Noise - Commercial        16804
Noise - Street/Sidewalk   16562
Derelict Vehicle          12081
Noise - Vehicle           11658
Vending                   3897
Traffic                   2870
Noise - Park              2182
Homeless Encampment       1734
Name: count, dtype: int64
```

5.2 Making a bar graph

```
In [23]: %matplotlib inline
df['Complaint Type'].value_counts()[:10].plot(kind='bar')
```

Out[23]: <Axes: xlabel='Complaint Type'>



6. Isolating columns with a certain value

```
In [24]: # Find all columns whose complaints are noise-residential
noise_res = df[ df['Complaint Type'] == 'Noise - Residential']
noise_res.head()
```

Out[24]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Location
5770	19832686	2/14/2011 5:28	2/14/2011 5:59	NYPD	New York City Police Department	Noise - Residential	Loud Music/Party	Residence Building/H
5771	20884743	7/23/2011 0:44	7/23/2011 2:50	NYPD	New York City Police Department	Noise - Residential	Loud Music/Party	Residence Building/H
5772	20893111	7/24/2011 1:33	7/24/2011 2:57	NYPD	New York City Police Department	Noise - Residential	Loud Music/Party	Residence Building/H
5773	20893310	7/23/2011 22:56	7/24/2011 0:03	NYPD	New York City Police Department	Noise - Residential	Loud Music/Party	Residence Building/H
5774	20897283	7/24/2011 3:18	7/24/2011 4:02	NYPD	New York City Police Department	Noise - Residential	Loud Music/Party	Residence Building/H

5 rows × 54 columns



In [25]:

```
# We can subset based on more than one column
# For example, if we want a data frame with all the noise-residential complaints in zi
noise_res = df[ (df['Complaint Type'] == 'Noise - Residential') & (df['Incident Zip']
noise_res.head()
```

Out[25]:

	Unique Key	Created Date	Closed Date	Agency	Agency Name	Complaint Type	Descriptor	Location
5771	20884743	7/23/2011 0:44	7/23/2011 2:50	NYPD	New York City Police Department	Noise - Residential	Loud Music/Party	Residence Building/H
5781	20944035	7/31/2011 23:03	8/1/2011 2:52	NYPD	New York City Police Department	Noise - Residential	Loud Music/Party	Residence Building/H

2 rows × 54 columns



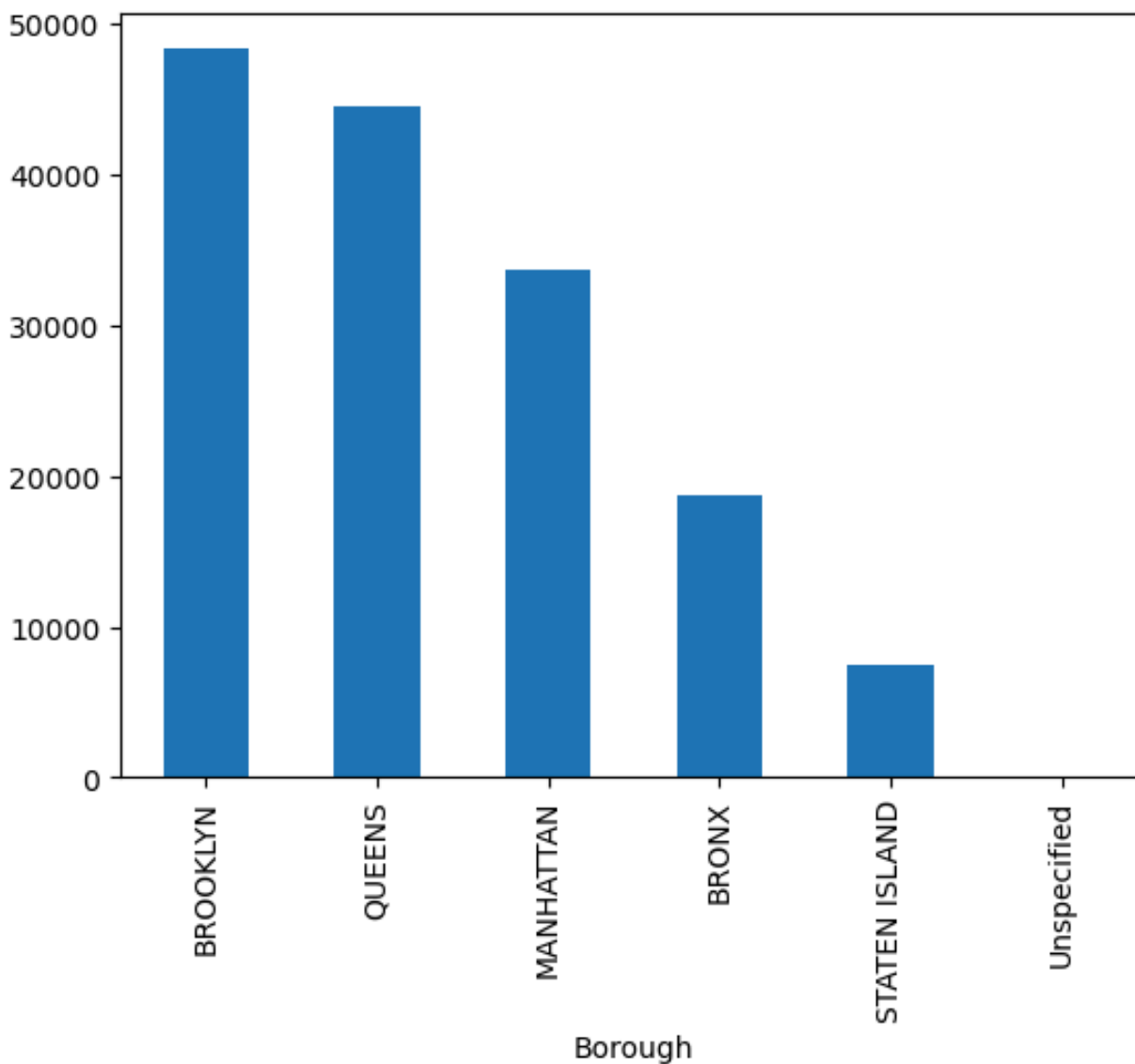
7. Bar Chart

In [26]: *# To see which Borough has the most 311 calls*

```
import matplotlib
%matplotlib inline

df['Borough'].value_counts().plot(kind='bar')
```

Out[26]: <Axes: xlabel='Borough'>



In [27]: `(df['Borough'].value_counts()/sum(df['Borough'].value_counts())).plot(kind='bar')`

Out[27]: <Axes: xlabel='Borough'>

